

权限控制模块的实现逻辑

Signature

Date

CONTENTS

目录

01

后端 Spring
Security 集成

02

前端复用 Axios

03

Cookie

04

前端 API 层实现

05

登录和注册前端
页面实现

01

后端 Spring Security 集成

集成 Spring Security

01

Maven 配置

配置 Maven ，在 pom.xml 添加依赖。

03

Spring Security 配置类

实现 Spring Security 配置类。

02

开发认证相关的类

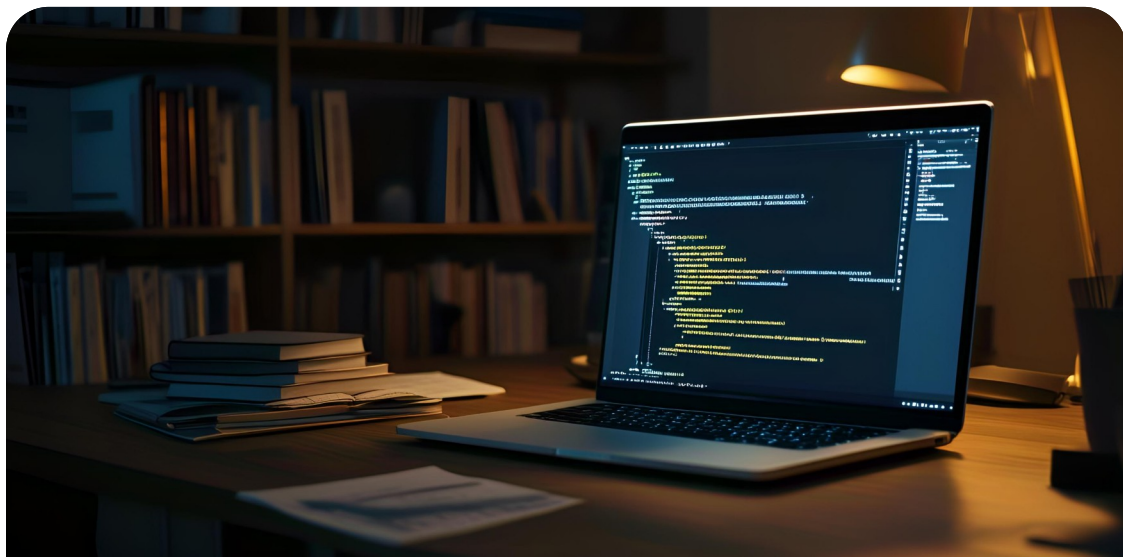
UserDetails 实现类、JWT 工具类、JWT 认证过滤器、UserDetailsService 实现类

04

实现用户的注册和登录逻辑

用户登录和注册的实现

依赖添加



Maven 依赖

在 pom.xml 添加 Spring Security 依赖,

```
<!-- Spring Security -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

```
<!-- JWT -->
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-api</artifactId>
    <version>0.11.5</version>
</dependency>
```

```
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-impl</artifactId>
    <version>0.11.5</version>
    <scope>runtime</scope>
</dependency>
```

```
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-jackson</artifactId>
    <version>0.11.5</version>
    <scope>runtime</scope>
</dependency>
```

开发权限认证相关的类

UserDetails 实现类



JWT 工具类



JWT 认证过滤器



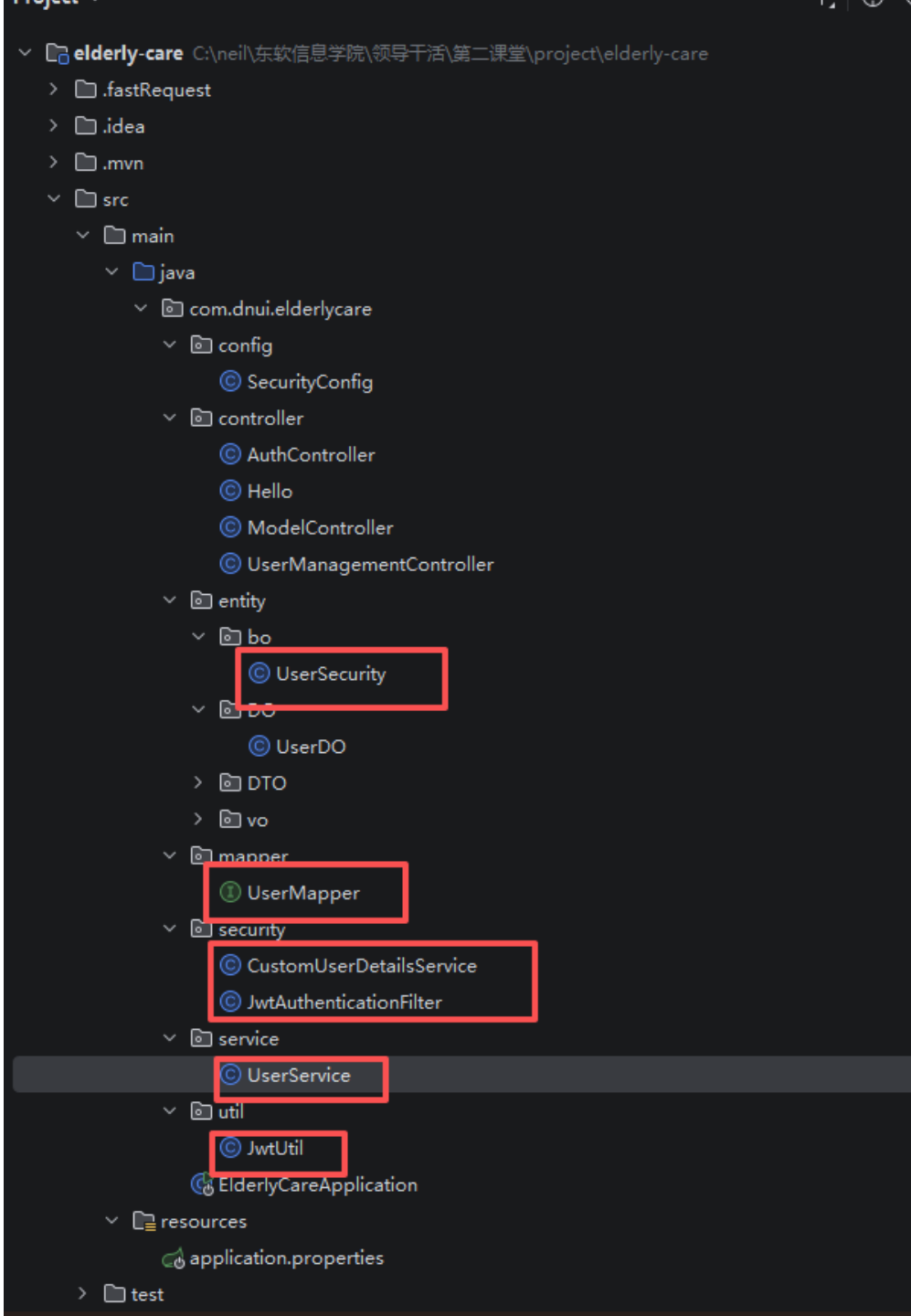
UserDetailsService 实现类



UserService 类



UserMapper 类

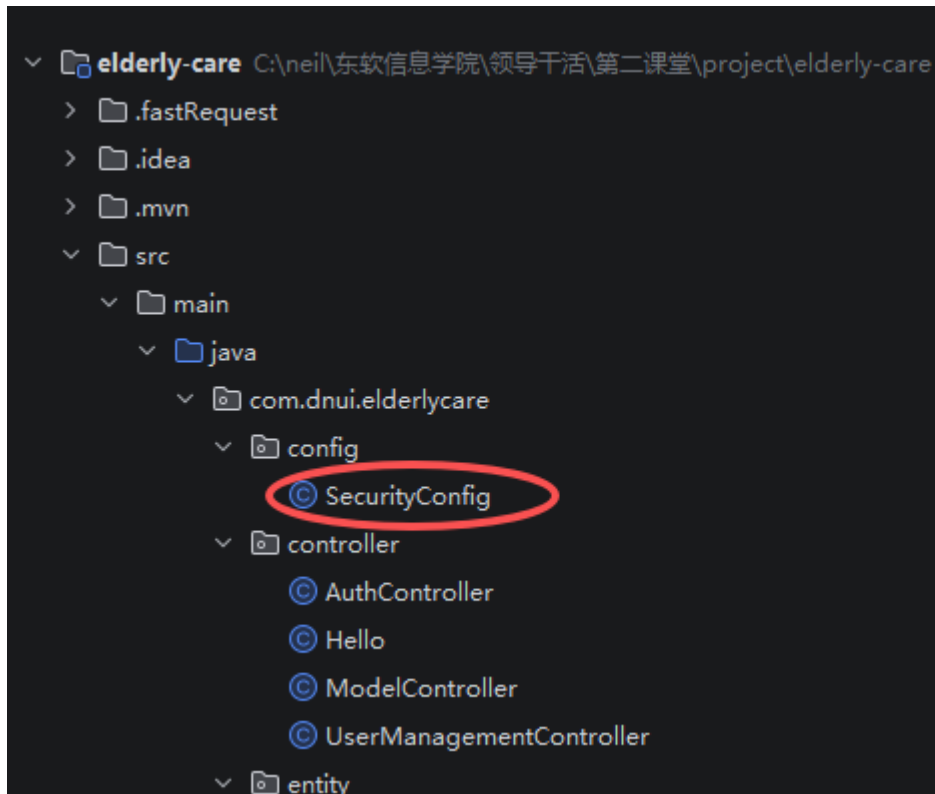


Spring Security 配置类

Spring Security 配置类



SecurityConfig.java



实现用户登录注册逻辑

登录和注册实现，以及
环境参数配置



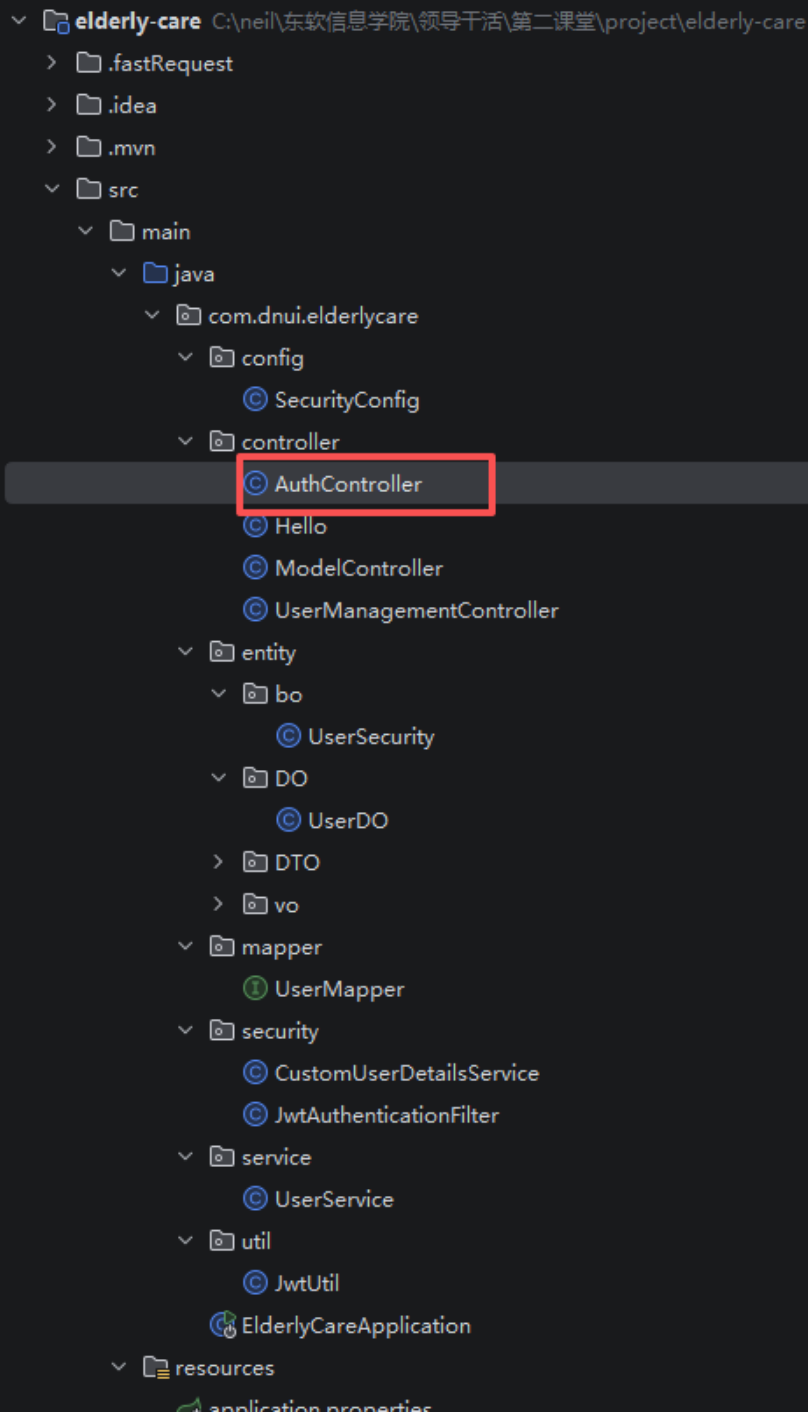
AuthController.java
va

```
1  spring.application.name=elderly-care
2
3  server.port = 8081
4
5  spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
6  spring.datasource.url=jdbc:mysql://localhost:3306/elderly-care
7  spring.datasource.username=root
8  spring.datasource.password=root
9
10 spring.datasource.type=com.alibaba.druid.pool.DruidDataSource
11 spring.datasource.initialSize=20
12 spring.datasource.minIdle=10
13 spring.datasource.maxActive=100
14
15 # JWT Configuration
16 jwt.secret=yourSecretKeyHereMakeItLongAndRandom
17 jwt.expiration=86400000
18
```

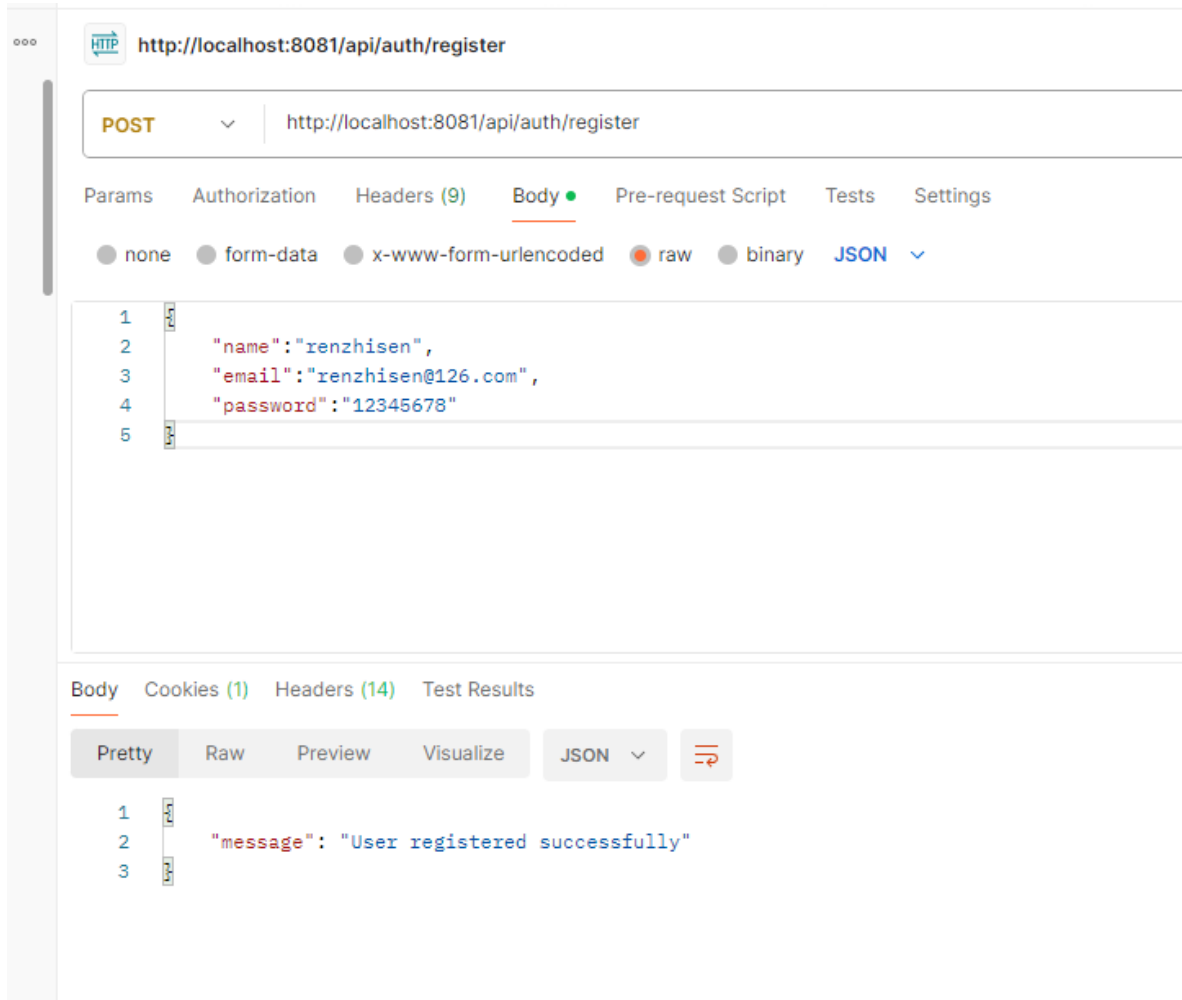
JWT Configuration

jwt.secret=yourSecretKeyHereMakeItLongAndRandom

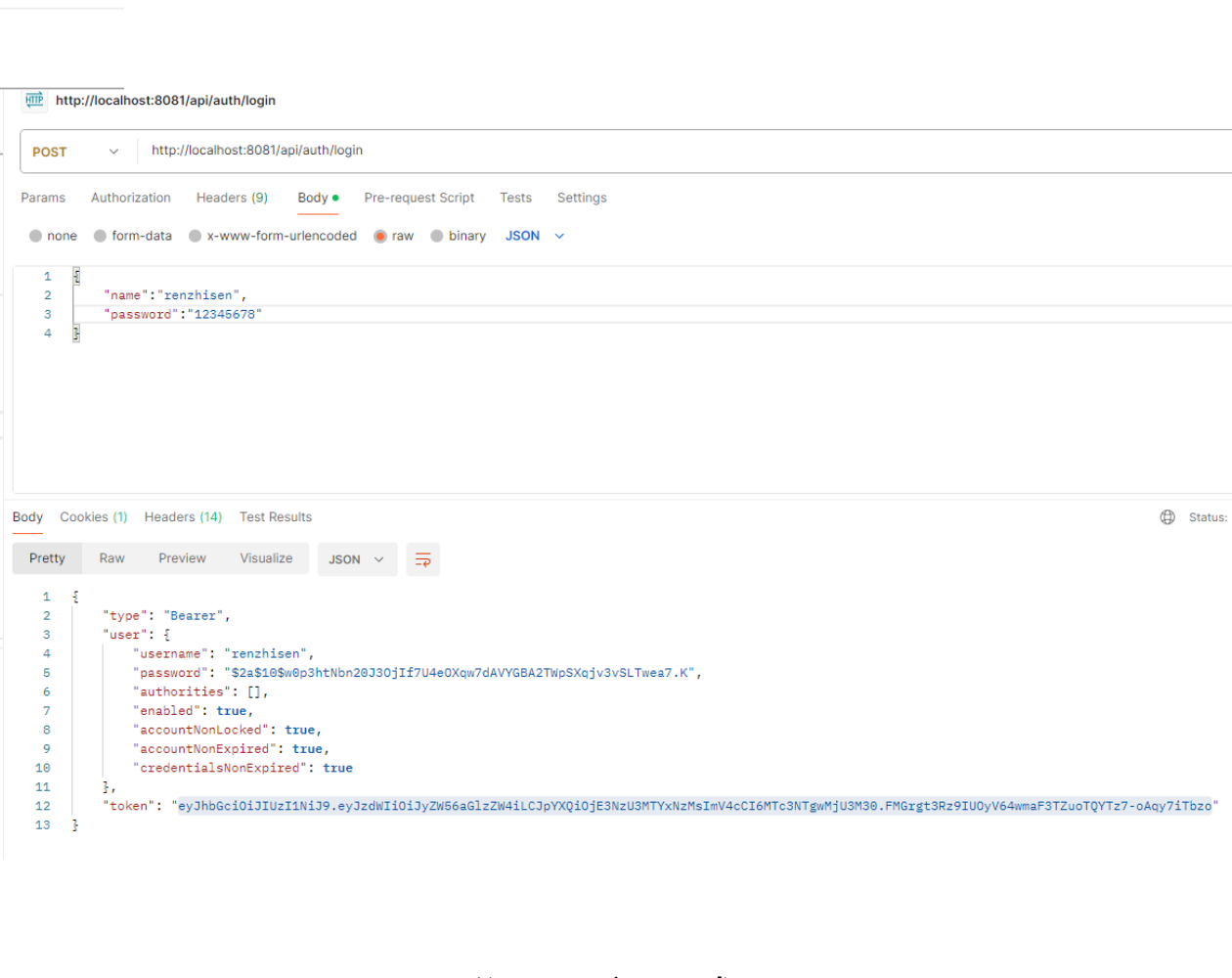
jwt.expiration=86400000



测试集成效果 - 使用 postman 测试后端接口



注册用户测试



登录用户测试

02

复用 Axios , 使用拦截器

复用 Axios

```
// utils/api.js
import axios from 'axios';

const httpRequest= axios.create({
  baseURL: '/api',
  timeout : 1000
  headers: {
    'Content-Type': 'application/json',
  },
});

export default httpRequest;
```

创建一个单例的 axios 对象，设置公共的后端访问 URL 和公共的 header 参数同时为后续的拦截器提供支持。

拦截器

通过拦截器，我们可以实现：

- 统一的认证处理：在请求拦截器中添加 token ，在响应拦截器中处理 401 未授权情况。
- 统一的错误处理：在响应拦截器中处理 HTTP 错误状态码，显示友好的错误提示。
- 加载状态管理：在请求开始时显示加载状态，在响应结束时隐藏加载状态。
- 响应数据预处理：对后端返回的数据格式进行统一处理，提取有效数据。

完整方案地址：

<https://www.oryoy.com/news/vue3-yu-axios-wan-meiji-cheng-zhi-nan-cong-jichu-pei-zhi-dao-gaoji-ying-yong-quanmian-ti-sheng-q.html>

请求拦截器

```
import axios from 'axios';

const httpRequrest= axios.create({
  baseURL: '/api',
  timeout : 1000
  headers: {
    'Content-Type': 'application/json',
  },
});

// 请求拦截器
httpRequrest.interceptors.request.use(
  config => {
    // 在发送请求之前做些什么
    return config
  },
  error => {
    // 对请求错误做些什么
  }
)

export default httpRequrest;
```

响应拦截器

```
import axios from 'axios';

const httpRequrest= axios.create({
  baseURL: 'api',
  timeout : 1000
  headers: {
    'Content-Type': 'application/json',
  },
});

// 响应拦截器
httpRequrest.interceptors.response.use(
  response => {
    // 对响应数据做点什么
  },
  error => {
    // 对响应错误做点什么
    return Promise.reject(error)
  }
)

export default httpRequrest;
```

样例



index.ts

EXPLORER

MY-VUE3-APP

> node_modules

> public

> src

> api

TS index.ts

TS user.ts

> assets

> components

> models

TS LoginResponse.ts

TS User.ts

> router

JS index.js

> utils

TS auth.ts

App.vue

TS main.ts

TS shims-vue.d.ts

.gitignore

babel.config.js

index.html

jsconfig.json

package-lock.json

src > api > TS index.ts > ...

1 import axios from 'axios'

2 import { ElMessage } from 'element-plus'

3 import { getToken, removeToken } from '@/utils/auth'

4 import router from '@/router'

5

6 const httpRequest = axios.create({

7 baseUrl: '/api',

8 timeout: 10000,

9 headers: {

10 'Content-Type': 'application/json',

11 },

12 })

13

14 // 请求拦截器 - 自动携带token

15 httpRequest.interceptors.request.use(

16 config => {

17 // 从cookie中获取token

18 const token = getToken()

19 if (token) {

20 config.headers['Authorization'] = 'Bearer ' + token

21 }

22 return config

23 },

24 error => {

25 console.error('请求错误:', error)

03

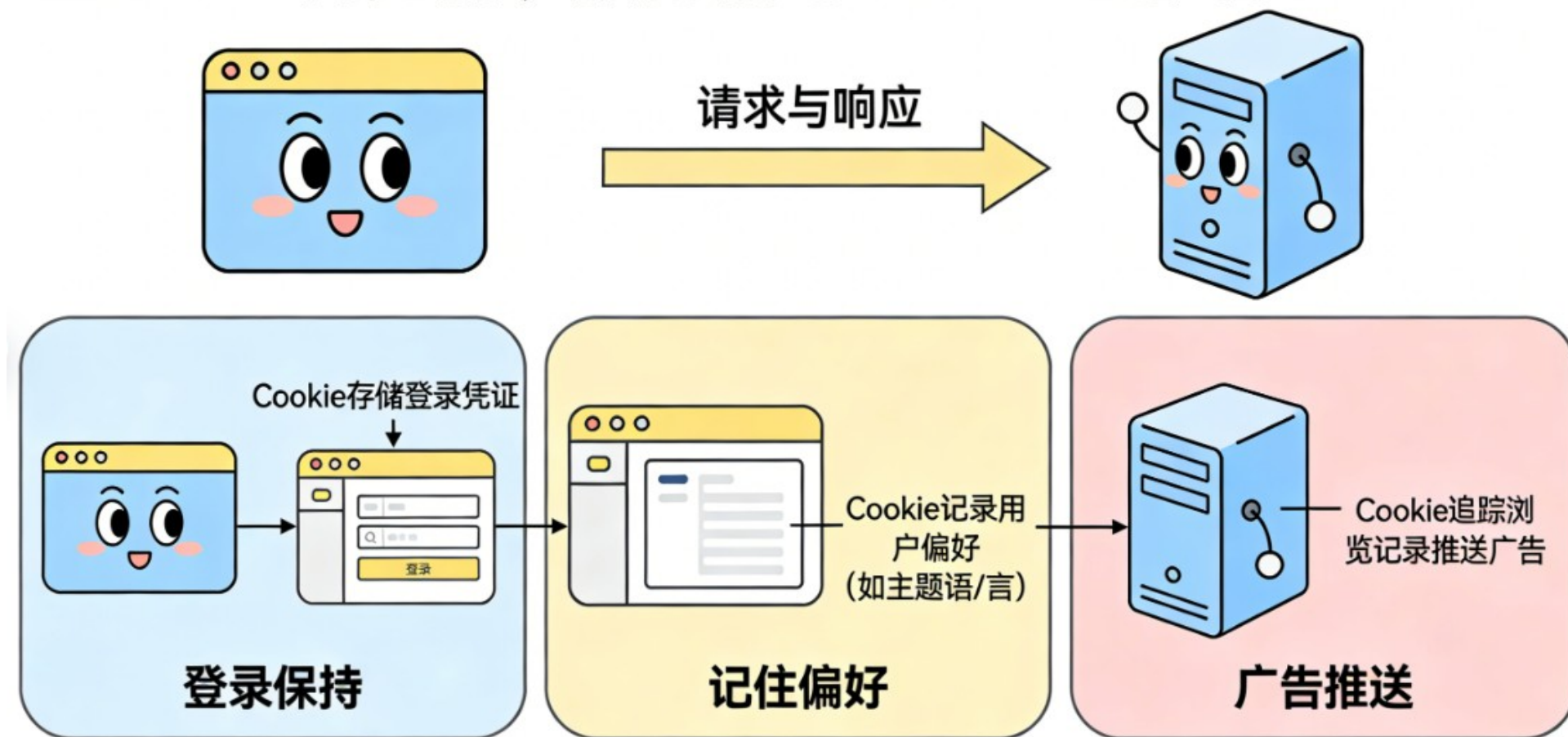
Cookie 存储 Token

什么是 Cookie

浏览器给网站开的一个小记事本，存在你电脑上，网站下次打开就能认出你

AI生成

浏览器、服务器与Cookie的关系

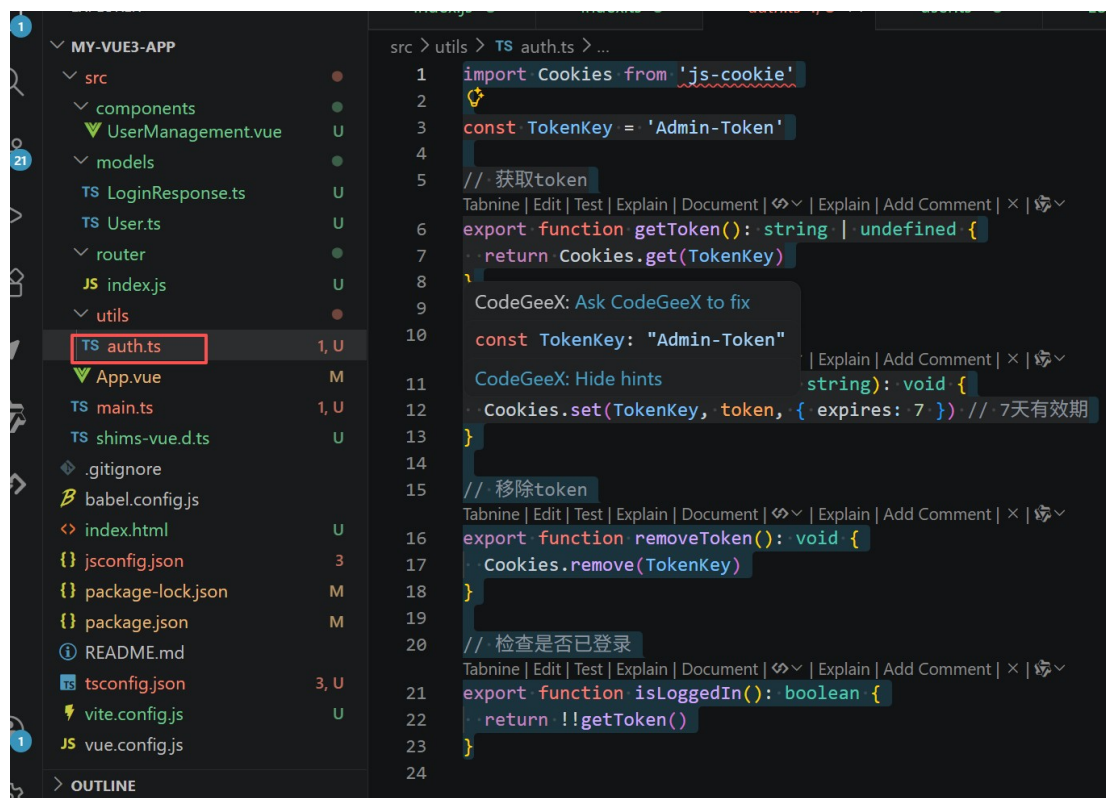


如何存储

添加js-cookies 依赖

`npm install js-cookie --save`

使用 cookie 操作 token



```
import Cookies from 'js-cookie'
```

```
const TokenKey='Admin-Token'
```

```
// 获取 token
export function getToken():string|undefined{
  return Cookies.get(TokenKey)
}
```

```
// 设置 token
export function setToken(token:string):void{
  Cookies.set(TokenKey, token, { expires:7 }) // 7 天有效期
}
```

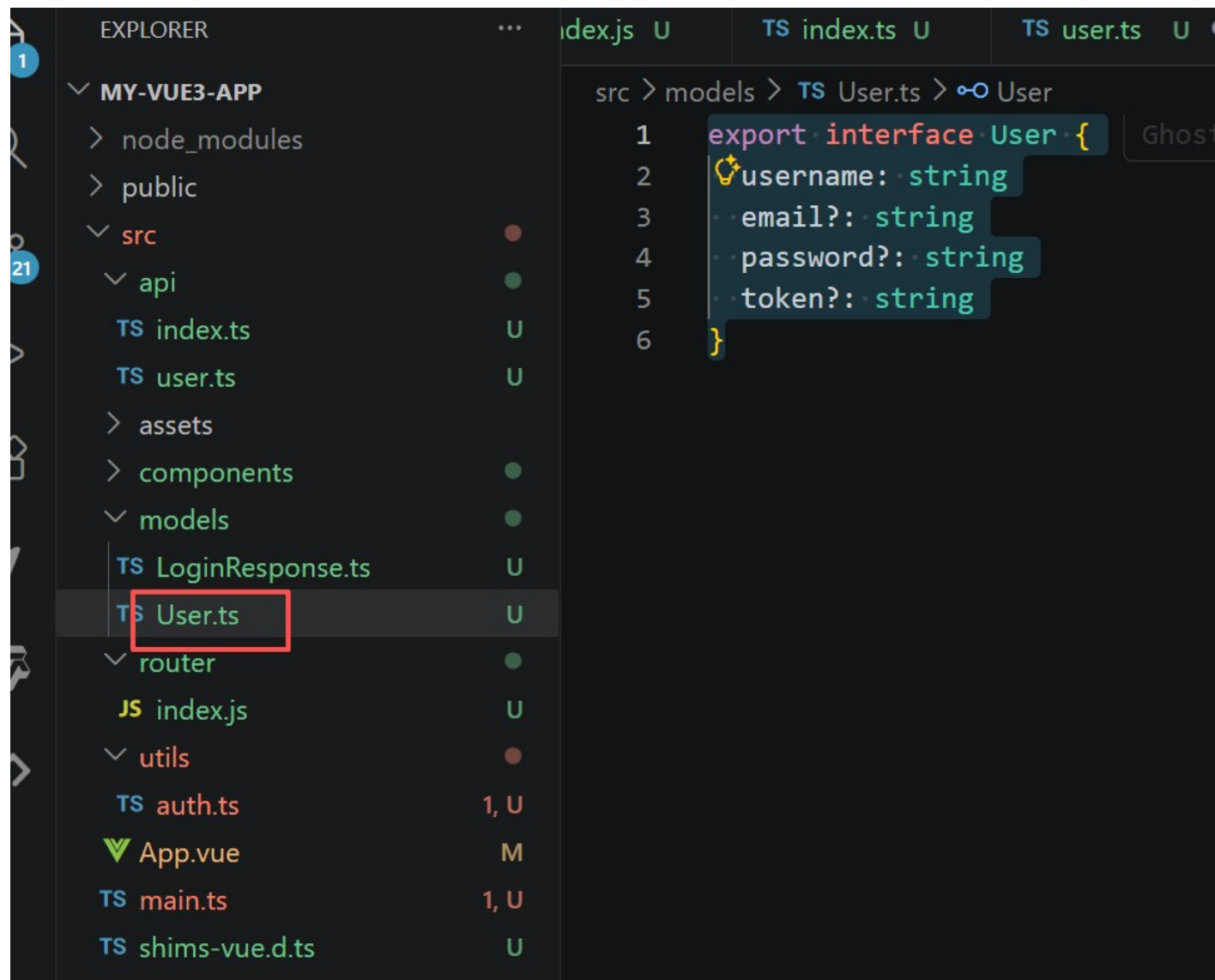
```
// 移除 token
export function removeToken():void{
  Cookies.remove(TokenKey)
}
```

```
// 检查是否已登录
export function isLoggedIn():boolean{
  return !!getToken()
}
```

04

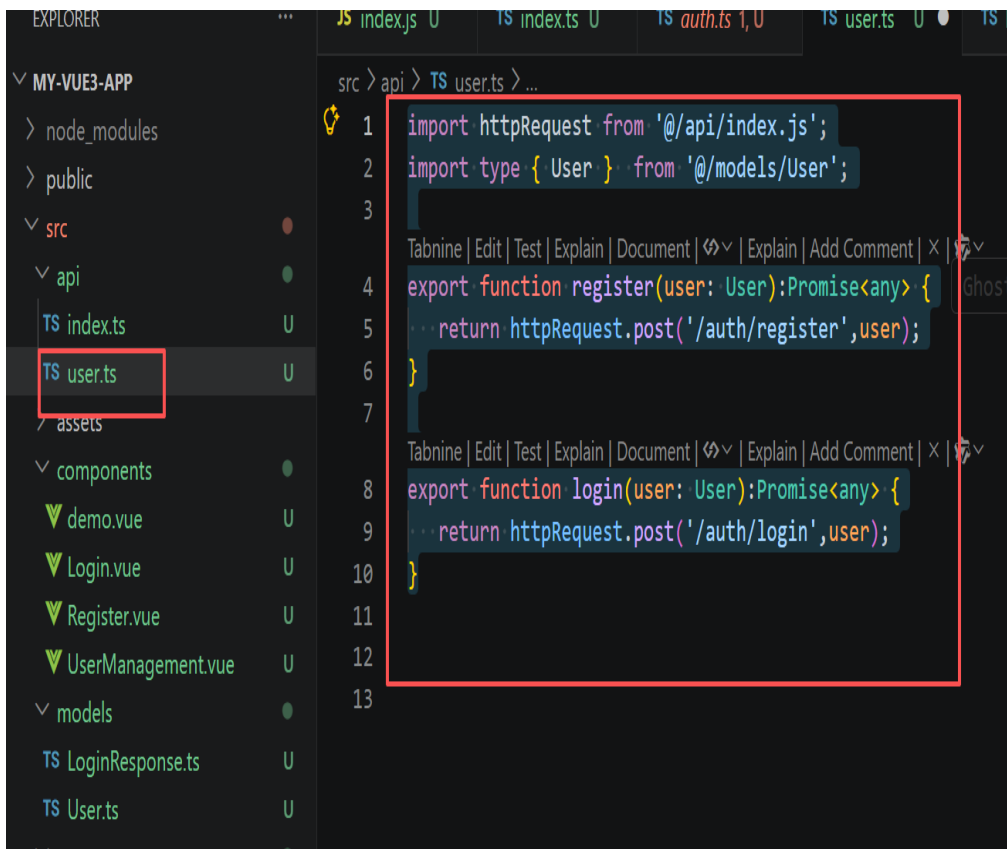
用户登录、注册等操作 API

用户对象数据模型的定义



```
export interface User {  
  username:string  
  email?:string  
  password?:string  
  token?:string  
}
```

前端构建用户登录、操作的 API 层



```
import httpRequest from '@api/index.js';
import type { User } from '@models/User';
```

```
export function register(user: User): Promise<any> {
  return httpRequest.post('/auth/register', user);
}
```

```
export function login(user: User): Promise<any> {
  return httpRequest.post('/auth/login', user);
}
```

05

用户登录、注册页面的开发

登录页面



Login.vue

用户登录

* 用户名

请输入用户名

* 密码

请输入密码

登录

还没有账号? [立即注册](#)

注册页面



Register.vue

用户注册

* 用户名

请输入用户名

* 邮箱

* 密码

* 确认密码

注册

已有账号? [立即登录](#)



THE END

谢谢观看